

西南科技大学计算机科学与技术学院

实验报告

实验名称 Linux 环境 C 程序开发

实验地点 东 11c225

实验日期 2024 年 11 月 14 日

指导教师 韦勇

学生班级

学生姓名

学生学号

提交日期 2024 年 11 月 21 日

一、实验目的

- 1.熟悉字符模式下的 C 程序编译和调试环境
- 2.基本掌握 Linux 字符模式下的编译工具和调试工具的使用
- 3.掌握自动编译工具 make 的使用和 Makefile 的编写方法

二、实验设计

- 1.GCC 使用方法
- 2.调试器 GDB 使用方法（选做）
- 3.编译配置工具 Make 工具使用
- 4.Linuxer 编译配置

三、实验记录

任务 1: 编译器 GCC 使用

使用本实验介绍的三种编译方法，对 root/workspace/.example 目录下的 bubbleSort.c, insertSort.c, selectSort.c, sort.c, sot.h 进行编译（其中，sot.c 是主程序，Sot.h 是头文件，其他的三个 c 程序分别实现了主程序需要用到的 3 个函数），写出步骤和结果。

1. 直接使用如下命令行一次完成编译、链接过程

先进入 root/workspace/.example 目录下，查看当前目录下文件。

```
[root@master ~]# cd /root/workspace/example/  
[root@master example]# ls  
bubbleSort.c  insertSort.c  Makefile      sort.c  
calc          make          selectSort.c  sort.h
```

输入 gcc bubbleSort.c insertSort.c selectSort.c sort.c sort.h -o main 指令对指定文件进行编译，发现新增 main 文件

```
[root@master example]# gcc bubbleSort.c insertSort.c selectSort.c  
sort.c sort.h -o main  
[root@master example]# ls  
bubbleSort.c  insertSort.c  make      selectSort.c  sort.h  
calc          main          Makefile  sort.c
```

输入 ./main 运行可执行程序

```

[root@master example]# ./main
the original list:
  13   0   5   8   1   7  21  50   9   2
after InsertionSort:
  0  13   5   8   1   7  21  50   9   2
  0   5  13   8   1   7  21  50   9   2
  0   5   8  13   1   7  21  50   9   2
  0   1   5   8  13   7  21  50   9   2
  0   1   5   7   8  13  21  50   9   2
  0   1   5   7   8  13  21  50   9   2
  0   1   5   7   8  13  21  50   9   2
  0   1   5   7   8   9  13  21  50   2
  0   1   2   5   7   8   9  13  21  50
  0   1   2   5   7   8   9  13  21  50
after SelectSort:
  0  13   5   8   1   7  21  50   9   2
  0   1  13   8   5   7  21  50   9   2
  0   1   2  13   8   7  21  50   9   5
  0   1   2   5  13   8  21  50   9   7
  0   1   2   5   7  13  21  50   9   8
  0   1   2   5   7   8  21  50  13   9
  0   1   2   5   7   8   9  50  21  13

```

2. 编译静态库，编译 main.c 时指定静态库位置。

为了避免跟下一种方法混淆，先输入 `rm main` 删除一下上一种方法生成的可执行文件。

```

[root@master example]# rm main
rm: 是否删除普通文件 "main"? y

```

输入 `gcc -c bubbleSort.c insertSort.c selectSort.c sort.c` 指令对指定文件进行编译生成中间文件 .o 文件。

`-c` 是一个常用选项，它的作用是只进行编译，不进行链接。也会将源代码文件（通常是 .c 文件）编译成目标文件（通常是 .o 文件），而不会生成最终的可执行文件。

```

[root@master example]# gcc -c bubbleSort.c insertSort.c
rt.c sort.c
[root@master example]# ls
bubbleSort.c  insertSort.c  Makefile      sort.c
bubbleSort.o  insertSort.o  selectSort.c  sort.h
calc          make          selectSort.o  sort.o

```

输入 `ar -r libcalc.a bubbleSort.o insertSort.o selectSort.o sort.o` 编译为静态库。

```

[root@master example]# ar -r libcalc.a bubbleSort.o insertSort.o
selectSort.o sort.o
ar: 正在创建 libcalc.a

```

输入 `gcc sort.c libcalc.a -o main` 生成 main 可运行程序

输入 `./main` 运行可执行程序。

```

[root@master example]# gcc sort.c libcalc.a -o main
[root@master example]# ./main
the original list:
    13    0    5    8    1    7    21    50    9    2
after InsertionSort:
    0    13    5    8    1    7    21    50    9    2
    0    5    13    8    1    7    21    50    9    2
    0    5    8    13    1    7    21    50    9    2
    0    1    5    8    13    7    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    9    13    21    50    2
    0    1    2    5    7    8    9    13    21    50
    0    1    2    5    7    8    9    13    21    50
after SelectSort:
    0    13    5    8    1    7    21    50    9    2
    0    1    13    8    5    7    21    50    9    2
    0    1    2    13    8    7    21    50    9    5
    0    1    2    5    13    8    21    50    9    7
    0    1    2    5    7    13    21    50    9    8
    0    1    2    5    7    8    21    50    13    9

```

3. 建立共享库。

首先，输入 `rm libcalc.a main` 删除上一种方法编译产生的文件，防止混淆。

```

[root@master example]# rm libcalc.a main
rm: 是否删除普通文件 "libcalc.a"? y
rm: 是否删除普通文件 "main"? y

```

输入 `gcc -shared bubbleSort.o insertSort.o selectSort.o sort.o -o calc.so` 命令使用 `bubbleSort.o insertSort.o selectSort.o sort.o` 生成共享库 `sort.so`。

```

[root@master example]# gcc -shared bubbleSort.o insertSort.o selectSort.o sort.o -o sort.so
[root@master example]# ls
bubbleSort.c  insertSort.c  Makefile      sort.c  sort.so
bubbleSort.o  insertSort.o  selectSort.c  sort.h
calc          make          selectSort.o  sort.o

```

输入 `gcc sort.c sort.so -o main` 生成可运行文件。

```

[root@master example]# gcc sort.c sort.so -o main
[root@master example]# ./main
./main: error while loading shared libraries: sort.so: cannot open shared object file: No such file or directory

```

报错：error while loading shared libraries: calc.so: cannot open shared object file: No such file or directory

原因是：共享库编译的 `main` 程序执行时需要加载共享库 `sort.so`，而步骤一和步骤二生成的共享库 `sort.so` 不在系统默认的共享库搜索路径中，所以错误提示为共享库 `sort.so` 不存在。可以通过如下命令指明共享库的搜索路径为 `sort.so` 所在的当前目录：

export LD_LIBRARY_PATH=`pwd`,再次运行新生成的该程序 main, 则可以正常运行。

```
[root@master example]# export LD_LIBRARY_PATH=`pwd`
[root@master example]# ./main
the original list:
    13    0    5    8    1    7    21    50    9    2
after InersionSort:
    0    13    5    8    1    7    21    50    9    2
    0    5    13    8    1    7    21    50    9    2
    0    5    8    13    1    7    21    50    9    2
    0    1    5    8    13    7    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    9    13    21    50    2
    0    1    2    5    7    8    9    13    21    50
after SelectSort:
    0    13    5    8    1    7    21    50    9    2
    0    1    13    8    5    7    21    50    9    2
    0    1    2    13    8    7    21    50    9    5
    0    1    2    5    13    8    21    50    9    7
    0    1    2    5    7    13    21    50    9    8
    0    1    2    5    7    8    21    50    13    9
```

任务 3: 编译工具 Make 使用

为/root/workspace/example/目录下的 C 代码 “bubbleSort.c, insertSort.c, sort.c, selectSort.c, sort.h” 编写 Makefile 文件, 要求至少包括 all 和 clean 两个目标, 且使用特定变量。

(1) 进入目录输入 vim Makefile 编写文件。

(2) 编辑 Makefile 文件为下图, 保存退出

```
all:mainapp
mainapp: sort.o insertSort.o selectSort.o bubbleSort.o
        gcc -o $@ $^
sort.o:sort.c sort.h
        gcc -c $<
inserSort.o:insertSort.c
        gcc -c $<
selectSort.o:selectSort.c
        gcc -c $<
bubbleSort.o:bubbleSort.c
        gcc -c $<
clean:
        rm mainapp sort.o insertSort.o selectSort.o bubbleSort.o
2>/dev/nul
```

(3) 进行编译, 输入 make -f Makefile。当前目录下多了 mainapp 文件。


```
[root@master example]# make -f Makefile
gcc -c sort.c
cc -c -o insertSort.o insertSort.c
gcc -c selectSort.c
gcc -c bubbleSort.c
gcc -o mainapp sort.o insertSort.o selectSort.o bubbleSort.o
[root@master example]# ls
bubbleSort.c  insertSort.c  make          selectSort.o  sort.o
bubbleSort.o  insertSort.o  Makefile      sort.c
calc          mainapp       selectSort.c  sort.h
```

(4) 运行 mainapp:

```
[root@master example]# ./mainapp
the original list:
    13    0    5    8    1    7    21    50    9    2
after InersionSort:
    0    13    5    8    1    7    21    50    9    2
    0    5    13    8    1    7    21    50    9    2
    0    5    8    13    1    7    21    50    9    2
    0    1    5    8    13    7    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    13    21    50    9    2
    0    1    5    7    8    9    13    21    50    2
    0    1    2    5    7    8    9    13    21    50
    0    1    2    5    7    8    9    13    21    50
after SelectSort:
    0    13    5    8    1    7    21    50    9    2
    0    1    13    8    5    7    21    50    9    2
    0    1    2    13    8    7    21    50    9    5
    0    1    2    5    13    8    21    50    9    7
    0    1    2    5    7    13    21    50    9    8
    0    1    2    5    7    8    21    50    13    9
```

(5) 使用 make -f Makefile clean, 发现文件被清理

```
[root@master example]# make -f Makefile clean
rm mainapp sort.o insertSort.o selectSort.o bubbleSort.o 2>/dev
ull
[root@master example]# ls
bubbleSort.c  insertSort.c  Makefile      sort.c
calc          make          selectSort.c  sort.h
```

任务 4: Linuxer 编译配置

请根据 Makefile 的写法, 对案例项目 Linuxer 的 Makefile 文件进行修改, 要求为:

(1) 使用变量, 为生成的*.tar.gZ 文件增加软件名称、版本号等信息; 注: 在 Makefile 中定义变量, 使用变量名-变量值, 使用变量: \$变量名。

将 linuxer.tar.gz 替换为\$(TAR_NAME), linuxer 替换为\$(SOFTWARE_NAME)

(2)实现 help 目标，用来提示列出 Makefile 中的所有目标；

```
SOFTWARE_NAME = linuxer
VERSION = 1.0.0
TAR_NAME=$(SOFTWARE_NAME)-$(VERSION).tar.gz

all: start_client

install: src_install

uninstall: src_uninstall

clean: src_clean

start_client: /opt/$(SOFTWARE_NAME)
    test -d /opt/$(SOFTWARE_NAME) && /opt/$(SOFTWARE_NAME)/bin/$(SOFTWARE_NAME).bash

help:
    echo "TODO"
    echo "all          编译并启动客户端"
    echo "install       安装软件"
    echo "uninstall      卸载软件"
    echo "clean          清理生成的文件"
    echo "start_client   启动客户端"
    echo "help           列出所有目标及说明"
    echo "src_install     安装源文件"
    echo "src_uninstall   卸载源文件"
    echo "src_clean       清理源文件"
    echo "$(TAR_NAME)  打包生成tar.gz文件"
    echo "tar_clean       清理tar.gz文件"

# source only
src_install: etc/* lib/* bin/* score doc
    test -d /opt/$(SOFTWARE_NAME) || mkdir -p /opt/$(SOFTWARE_NAME) >/dev/null 2>&1
    test -d /opt/$(SOFTWARE_NAME)/score || mkdir -p /opt/$(SOFTWARE_NAME)/score >/dev/null 2>&1
    cp -rf etc/ lib/ bin/ doc/ labs/ /opt/$(SOFTWARE_NAME) >/dev/null 2>&1
src_uninstall: /opt/$(SOFTWARE_NAME)
    rm -rf /opt/$(SOFTWARE_NAME)
src_clean:
    rm deb/opt -rf >/dev/null 2>&1

# tar.gz
$(TAR_NAME): tar/
    test -d tar/$(SOFTWARE_NAME) || mkdir -p tar/$(SOFTWARE_NAME) >/dev/null 2>&1
    cp -rf etc/ bin/ doc/ labs/ lib/ score/ CHANGELOG LICENSE Makefile tar/$(SOFTWARE_NAME)
    >/dev/null 2>&1
    cd tar/ ; tar -cvzf ../$(TAR_NAME) $(SOFTWARE_NAME)/ >/dev/null 2>&1

tar_clean:
    rm $(TAR_NAME) >/dev/null 2>&1
    rm tar/$(SOFTWARE_NAME) -rf >/dev/null 2>&1
```

1 添加软件名称、版本号等信息

2 实现help，列出所有目标

3 源文件中linuxer都替换为\$(SOFTWARE_NAME)
linuxer.tar.gz都替换为\$(TAR_NAME)

(2) 使用\$@, \$, \$<等变量，简化 Makefile 文件。

如图所示，对文件进行简化。其中：

\$@: 表示目标文件的名称。在 target: prerequisites 中，\$@代表 target。

\$<: 表示第一个依赖文件的名称。

\$^: 表示所有依赖文件的名称。

\$\$: 表示所有比目标文件更新的依赖文件，用于增量构建。

*: 表示不包含后缀的目标文件名，通常和模式规则一起使用。

```
VERSION = 1.0.0
INSTALL_DIR = /opt/$(SOFTWARE_NAME)
TAR_DIR = tar/linuxer
TAR_FILE = $(SOFTWARE_NAME)-$(VERSION).tar.gz
SOURCE_DIRS = etc lib bin score doc labs
SRC_FILES = $(SOURCE_DIRS) CHANGELOG LICENSE Makefile
```

定义变量

```
all: start_client
start_client: $(INSTALL_DIR)
    @test -d $@ && $@/bin/linuxer.bash
install: src_install
uninstall:
    rm -rf $@
clean: src_clean
```

```
help:
    echo "Makefile 目标:"
    echo "  all          - 启动客户端"
    echo "  install      - 安装软件"
    echo "  uninstall    - 卸载软件"
    echo "  clean        - 清理构建环境"
    echo "  help         - 显示帮助信息"
    echo "  $(TAR_FILE)  - 创建软件的 tarball 文件"
```

help目标

```
src_install: $(INSTALL_DIR)
    mkdir -p $@/score
    cp -rf $^ $@
```

进行简化

```
src_clean:
    rm -rf deb/opt $(TAR_FILE) $(TAR_DIR)
```

```
$(TAR_FILE): $(TAR_DIR)
    mkdir -p $(@D)
    cp -rf $^ $(@D)
    cd tar && tar -cvzf ../$(@) linuxer/
```

```
tar_clean:
    rm -f $(@) $(TAR_DIR)
```

\$(@D) 表示提取目标文件名中的目录部分，即 \$(@D) 代表 \$(TAR_FILE) 中的路径部分，不包括文件名。

进入 root/tools/文件夹下将 linuxer 解压，输入 tar -zxvf linuxer.tar.gz 压缩后进入 linuxer。

```
[root@master example]# cd /root/tools/
[root@master tools]# ls
example.tar.gz          tripwire-2.4.3.7-1.el7.x86_64.rpm
fping_4.0.orig.tar.gz  wput-0.6.tgz
linuxer.tar.gz
[root@master tools]# tar -zxvf linuxer.tar.gz
linuxer/
```

输入 vim Makefile 进行编辑文件，编辑为上面的内容后保存退出。

输入 make help

- 跨团队协作标准化：在团队开发中，通过提供一个标准化的 **Makefile**，所有开发者都能用一致的方法构建项目，避免因环境或习惯差异导致的问题。